# INDEX GENERATION PROJECT
# DSA LAB — CODING PROBLEMS WITH ANSWERS

Problems and solutions only. Based on your Python project using Trie, BST and Dictionary.

Code snippets show the intended change. Match class/variable names to your submitted file, and run tests after editing.

## 1. Add ! and ? as separators

Question: Update the delimiter pattern so ! and ? split words too.

```
DELIMITERS = re.compile(r"[\s.,:;!?]+")
```

Note: Replace the current DELIMITERS line. Keep the other separators.

## 2. Search for a word entered by the user

Question: After building the index, ask the user for a word and display its line numbers, or 'Word not found'.

```
query = input("Enter word: ").strip()
result = index.search(query)
if result is None:
    print("Word not found")
else:
    print("Word found on lines:", ", ".join(map(str, result)))
```

Note: Place this after the index has been built. This uses the existing search() method.

## 3. Show frequency with each indexed word

Question: Add the total number of occurrences to every output line.

```
for word, line_numbers in index.items_sorted():
    frequency = len(line_numbers)
    output.write(
        f"{word}\t{', '.join(map(str, line_numbers))}\tFrequency: {frequency}\n"
    )
```

Note: Use this pattern inside write_index(), replacing its current output.write line. Duplicates count as separate occurrences.

## 4. Make indexing case-insensitive

Question: Treat Apple, APPLE and apple as the same word.

```
if word:
    word = word.lower()
    yield word, line_number
```

Note: Put this in words_with_line_numbers() before yield. Also convert search input to lowercase before searching. Update tests because the original project preserves case.

## 5. Display only words that occur more than once

Question: Modify output writing so only repeated words appear.

```
for word, line_numbers in index.items_sorted():
    if len(line_numbers) &gt; 1:
        output.write(f"{word}\t{', '.join(map(str, line_numbers))}\n")
```

Note: Place the condition inside write_index(). A word appearing twice on one line also counts as repeated.

## 6. Search for a prefix in a Trie

Question: Add a Trie method that checks whether any word starts with a given prefix.

```
def has_prefix(self, prefix):
    node = self.root
    for char in prefix:
        if char not in node.children:
            return False
        node = node.children[char]
    return True
```

Note: Use the Trie root and child traversal. This checks the path, not whether the prefix itself is a complete word. Adjust self.root if your class uses a different root attribute name.

## 7. Count distinct words

Question: Print the number of unique words in the index.

```
# For DictionaryIndex:
print("Distinct words:", len(index.data))

# Structure-independent option:
distinct_count = sum(1 for _ in index.items_sorted())
print("Distinct words:", distinct_count)
```

Note: len(index.data) works only for DictionaryIndex. Use items_sorted() for Trie/BST too.

## 8. Use TrieIndex instead of DictionaryIndex

Question: Change the main program to build the index using TrieIndex.

```
index = build_index(TrieIndex, input_filename)
```

Note: Replace the DictionaryIndex class argument in the build_index call. If main() uses len(index.data), replace that count with sum(1 for _ in index.items_sorted()).

## 9. Use BSTIndex instead of DictionaryIndex

Question: Change the main program to build the index using BSTIndex.

```
index = build_index(BSTIndex, input_filename)
```

Note: Replace the class argument. Remember BSTIndex does not have DictionaryIndex's data attribute.

## 10. Keep duplicate line numbers

Question: For input `apple apple` on line 1, ensure search('apple') returns [1, 1].

```
self.data.setdefault(word, []).append(line_number)
```

Note: Use append for every occurrence. Do not use a set or remove repeated line numbers.

## 11. Handle a missing word safely

Question: Display a message when search returns no result.

```
result = index.search(query)
if result is None:
    print("Word not found")
else:
    print(result)
```

Note: All three structures return None for a missing word.

## 12. Change line numbering to start at 1

Question: Ensure the first input line is numbered 1.

```
for line_number, line in enumerate(file, start=1):
```

Note: Use start=1 in enumerate().

## 13. Add a new word and line number

Question: Ask the user for a word and line number, then add it to the index.

```
word = input("Enter word: ").strip()
line_number = int(input("Enter line number: "))
index.add(word, line_number)
print("Word added")
```

Note: Place after index construction if the task asks for an interactive update. Validate line_number if the question requires positive integers.

## 14. Sort output alphabetically

Question: Ensure words are written in sorted order.

```
for word, line_numbers in index.items_sorted():
    ...
```

Note: Use the existing items_sorted() interface. DictionaryIndex sorts keys; BST can use in-order traversal; Trie can traverse its keys.

## 15. Print line numbers as comma-separated values

Question: Convert [1, 2, 2, 4] to `1, 2, 2, 4` in output.

```
', '.join(map(str, line_numbers))
```

Note: map(str, ...) converts integers to strings before join().

### 16. Count occurrences of each word using a dictionary

Question: Build a frequency dictionary from a list of words.

```
frequency = {}
for word in words:
    frequency[word] = frequency.get(word, 0) + 1
```

Note: get(word, 0) gives zero for a new word, then the count increases by one.

### 17. Fix a failure after switching to Trie/BST

Question: The program raises AttributeError because index.data does not exist. Fix it.

```
distinct_count = sum(1 for _ in index.items_sorted())
print("Distinct words:", distinct_count)
```

Note: Only DictionaryIndex has a data dictionary. items_sorted() is shared by all three structures.

### 18. Add a file-not-found error message

Question: Prevent a traceback if the input file does not exist.

```
try:
    index = build_index(DictionaryIndex, input_filename)
except FileNotFoundError:
    print("Input file not found")
    return
```

Note: Put the try/except around the build_index call inside main(). Return from main after printing the message.

### 19. Write output to a different filename

Question: Run the program using input.txt and result.txt.

```
python index_generation.py input.txt result.txt
```

Note: The second filename is the output path. The program writes the index there.

### 20. Run the supplied tests

Question: Run the project unit tests from the project folder.

```
python -m unittest -v test_index_generation.py
```

Note: Check that the final test result is OK. If you intentionally changed case behavior, update the expected test output first.

### 21. Create a simple test for duplicate words

Question: Write a test confirming apple on line 1 twice and line 2 once returns [1, 1, 2].

```
index = DictionaryIndex()
index.add("apple", 1)
index.add("apple", 1)
index.add("apple", 2)
assert index.search("apple") == [1, 1, 2]
```

Note: This checks that every occurrence is retained.

### 22. Explain why sorted BST output works

Question: Implement sorted traversal for a BST.

```
def inorder(node):
    if node is not None:
        yield from inorder(node.left)
        yield node.word, node.line_numbers
        yield from inorder(node.right)
```

Note: This is recursive in-order traversal: left, node, right. Adapt the method to the node/class names in your code.

### 23. Measure build time

Question: Measure how long index construction takes.

```
from time import perf_counter
start = perf_counter()
```

```
index = build_index(DictionaryIndex, input_filename)
build_seconds = perf_counter() - start
print("Build time:", build_seconds)
```

Note: Start the timer immediately before building and subtract start immediately after.

## 24. Add a total occurrence count

Question: Print total word occurrences as well as distinct words.

```
total_occurrences = sum(
    len(line_numbers) for _, line_numbers in index.items_sorted()
)
print("Total occurrences:", total_occurrences)
```

Note: Distinct words count unique keys; total occurrences sums the lengths of all line-number lists.

## 25. Add a guard for an empty search query

Question: If the user enters nothing, ask them to enter a word instead of searching.

```
query = input("Enter word: ").strip()
if not query:
    print("Please enter a word")
else:
    result = index.search(query)
    if result is None:
        print("Word not found")
    else:
        print("Word found on lines:", ", ".join(map(str, result)))
```

Note: strip() removes surrounding spaces. Check the empty string before calling search().